

Table of contents

Chapitre 9 : Transitions de Phase en Optimisation	1
Introduction	1
Le Problème de Satisfiabilité (SAT)	2
Le Problème XORSAT	2
Algorithme pour Résoudre XORSAT	3
Problèmes k-XORSAT	4
Transition de Phase	4
Modèle Statistique : Première Approche	5
Modèle Statistique : Seconde Approche	5
Modèle Statistique	6
Espace des Solutions	6
Le Cas du 1-XORSAT	6
Problème 2-XORSAT	8
Cas du Problème 3-XORSAT	10
Comportement d'une Métaheuristique de Recherche	12
L'Algorithme Random Walk SAT (RWSAT)	12
Comportement de la Métaheuristique : Régime des petits α	13
Comportement de la Métaheuristique : Régime des grands α	13
Résumé du Comportement de la Métaheuristique	13
Complexité Algorithmique Linéaire et Exponentielle	14
Relation entre $e_{plateau}$ et τ_{res} en fonction de α	14
Algorithme de Retour Arrière (Backtracking)	14

Chapitre 9 : Transitions de Phase en Optimisation

Introduction

Nous cherchons à analyser statistiquement le comportement des métaheuristiques pour des problèmes d'optimisation de difficulté croissante.

Nous considérons une métaheuristique qui cherche une solution à des problèmes dits de satisfiabilité (SAT) en logique booléenne :

- La difficulté peut être ajustée au moyen d'un paramètre de contrôle continu.
- Ces problèmes peuvent être analysés théoriquement.
- On peut définir la probabilité qu'une instance donnée ait des solutions.
- On peut étudier si la métaheuristique sélectionnée peut la trouver.

Lorsque la difficulté augmente :

Nous observons que nous passons d'une situation où de nombreuses solutions sont trouvées à une situation sans aucune solution.

Nous verrons que la métaheuristique sélectionnée passe d'une complexité linéaire (en moyenne) à une complexité exponentielle.

Ce processus est appelé une **transition de phase**, en référence aux transitions de phase en physique (eau liquide à glace, ou matériaux magnétiques).

Ce comportement complexe provient de la façon dont les solutions sont organisées dans l'espace de recherche.

Le Problème de Satisfiabilité (SAT)

Un problème SAT est composé de N variables booléennes contraintes par M équations booléennes.

Les problèmes SAT forment une classe canonique de problèmes de décision en logique propositionnelle.

Le but est de trouver une affectation des N variables qui résout les M équations.

Ou plus généralement, une affectation qui minimise le nombre de contraintes non satisfaites.

Le nombre d'équations non satisfaites est appelé l'**Énergie** E .

Nous cherchons donc à minimiser E .

Le problème est dit **satisfiable** si on peut avoir $E = 0$.

Le Problème XORSAT

Le problème XORSAT est un cas particulier pour lequel les M équations de contrainte linéaires ne contiennent que des XOR.

Nous avons un problème linéaire en arithmétique modulo 2.

Exemple :

Problème XORSAT avec $M = 3$ contraintes et $N = 4$ variables.

$$\begin{cases} x_1 + x_2 + x_3 = 1 \\ x_2 + x_4 = 0 \\ x_1 + x_4 = 1 \end{cases}$$

Le symbole $+$ désigne l'addition **modulo 2** (c'est-à-dire XOR) en logique.

Les x_i sont des variables booléennes telles que $x_i \in \{0, 1\}$.

Ce problème est **satisfiable** (il admet une ou plusieurs solutions).

Par inspection, il est facile de trouver qu'il n'y a que deux solutions :

$$(x_1, x_2, x_3, x_4) = \begin{cases} (1, 0, 0, 0) \\ (0, 1, 0, 1) \end{cases}$$

On peut facilement construire un problème qui n'admet pas de solution.

Par exemple :

$$\begin{cases} x_1 + x_2 = 1 \\ x_1 + x_2 = 0 \end{cases}$$

Ce problème ne peut pas être satisfait puisque les deux contraintes ne peuvent pas être simultanément satisfaites.

Dans ce cas, le nombre maximum de contraintes satisfaites est 1 : $E = 1$.

Algorithme pour Résoudre XORSAT

Un problème XORSAT peut être résolu par une **élimination de Gauss** (en arithmétique modulo 2), en un temps $O(N^3)$, si $N = M$ (matrice carrée).

En d'autres termes : tous les problèmes XORSAT sont dans P (temps polynomial).

Il n'est donc **PAS difficile** de résoudre un problème XORSAT, contrairement aux problèmes SAT non linéaires qui peuvent contenir n'importe quel opérateur logique tel que AND, OR et NOT.

Élimination de Gauss : un Exemple

Considérons un système 3-XORSAT généré aléatoirement, avec $N = 10$ variables et $M = 8$ équations.

Après transformation par élimination de Gauss, le système peut être reformulé avec 8 variables contraintes dont les valeurs sont déterminées par deux variables libres x_6 et x_9 .

Ces variables libres peuvent prendre n'importe quelle valeur dans $\{0, 1\}$, donc 4 solutions possibles existent.

Représentation des solutions :

But : représenter et énumérer toutes les solutions de manière concise. Représentation “décimale” des solutions.

- $x_6x_9 = 0 \rightarrow x_3x_4x_5x_0x_1x_8x_2x_7 = 108$
- $x_6x_9 = 1 \rightarrow x_3x_4x_5x_0x_1x_8x_2x_7 = 193$
- $x_6x_9 = 2 \rightarrow x_3x_4x_5x_0x_1x_8x_2x_7 = 148$
- $x_6x_9 = 3 \rightarrow x_3x_4x_5x_0x_1x_8x_2x_7 = 57$

Problèmes k-XORSAT

Nous nous limitons à une sous-classe particulière de problèmes XORSAT : les problèmes dits **k-XORSAT**.

Les M équations représentant le système contiennent toutes exactement k variables parmi les N .

Notre objectif est d’analyser statistiquement l’espace des solutions des problèmes k-XORSAT pour différentes valeurs de k .

Méthode :

→ Générer aléatoirement de nombreux problèmes k -XORSAT et déduire la probabilité $P_{SAT}(N, M, k)$ qu’une instance donnée admette une solution.

En pratique, nous introduisons une nouvelle variable α définie par

$$\alpha = \frac{M}{N}$$

Nous considérons la probabilité de satisfiabilité P_{SAT} comme une fonction de N et α : $P_{SAT}(N, \alpha)$ pour des valeurs fixes données de k .

Lorsque α augmente, il devient **de plus en plus difficile** de satisfaire le problème (puisque le nombre de contraintes augmente).

Nous anticipons donc que $P_{SAT}(N, \alpha)$ diminue avec α .

Transition de Phase

Si $N \rightarrow \infty$ et $k \geq 2$, nous observons que $P_{SAT}(N, \alpha)$ subit une **transition de phase** pour une valeur critique $\alpha_c(k)$ de α .

Pour $\alpha < \alpha_c$: tous les problèmes sont satisfiables.

Pour $\alpha > \alpha_c$: aucun ne l’est.

Modèle Statistique : Première Approche

Pour construire des instances du problème k -XORSAT, avec N et α donnés, nous pouvons suivre cette procédure :

- Sélectionner uniformément et aléatoirement k indices distincts parmi N et un booléen ν .
- Générer l'équation suivante :

$$x_{i_1} + x_{i_2} + \dots + x_{i_k} = \nu$$

où $i_1, i_2, \dots, i_k$ sont les k indices sélectionnés au hasard.

Répéter ce processus $M = \alpha N$ fois et nous obtenons une instance particulière du problème avec exactement M contraintes.

Il est possible que plusieurs équations de contrainte soient identiques ou contradictoires.

Modèle Statistique : Seconde Approche

Une autre approche pour générer notre échantillon statistique consiste à **systématiquement** considérer **toutes** les équations possibles avec k variables parmi N .

→ Au lieu de générer exactement M équations une par une (comme dans la première approche), nous : 1. Énumérons toutes les équations k -XOR distinctes possibles sur N variables. 2. Sélectionnons chaque équation indépendamment avec une probabilité p .

C'est un ensemble classique de Bernoulli (arêtes indépendantes), analogue aux graphes aléatoires d'Erdős-Rényi.

→ Comme il y a deux façons possibles de choisir le terme de droite, il y a

$$H = 2 \binom{N}{k}$$

équations possibles.

→ Nous les inspectons toutes, et nous les sélectionnons avec une probabilité $p = M/H$, de sorte qu'à la fin de ce processus, il y ait approximativement $pH = M$ équations dans le système.

Modèle Statistique

La probabilité P_{SAT} est définie par rapport à l'approche statistique sélectionnée pour générer les problèmes k -XORSAT.

Pour les deux approches, nous répétons le processus un nombre $\mathcal{N} \gg 1$ suffisamment élevé pour générer une population de problèmes suffisamment grande pour échantillonner l'espace de tous les problèmes possibles.

Par exemple, nous choisissons $\mathcal{N} = 10'000$.

Nous résolvons (en utilisant l'élimination de Gauss) toutes ces $\mathcal{N}$ instances pour voir si elles sont satisfiables ou non.

La probabilité P_{SAT} est estimée comme la fraction de problèmes satisfiables parmi le total $\mathcal{N}$.

Espace des Solutions

Notre objectif est de caractériser l'espace des solutions pour les problèmes k -XORSAT, avec $k = 1, 2, 3$.

Pour ces valeurs de k , nous pouvons obtenir analytiquement plusieurs résultats démontrant l'existence de cette transition de phase.

Lorsque les calculs sont trop complexes mathématiquement, nous ne donnerons que le résultat sans plus de détails.

Le Cas du 1-XORSAT

Une instance du problème 1-XORSAT est par exemple :

$$\begin{cases} x_{i_1} = \nu_1 \\ \dots \\ x_{i_M} = \nu_M \end{cases}$$

Un tel système est satisfiable si aucune des x_i n'apparaît plus d'une fois dans les contraintes.

Si une variable apparaît plus d'une fois, alors le terme de droite doit être identique à chaque fois.

Calcul de P_{SAT} (1/4)

Nous avons :

$$P_{SAT} > P$$

où P est la probabilité qu'en tirant M équations, nous ne tirions jamais la même variable deux fois.

$$P = 1 \times \left(1 - \frac{1}{N}\right) \times \left(1 - \frac{2}{N}\right) \cdots \times \left(1 - \frac{M-1}{N}\right) = \prod_{i=0}^{M-1} \left(1 - \frac{i}{N}\right)$$

Ce problème est identique à celui de trouver la probabilité que parmi M personnes, aucune n'ait son anniversaire le même jour (dans ce cas $N = 365$ jours).

Calcul de P_{SAT} (2/4)

La probabilité P peut être estimée en prenant le logarithme :

$$\log P = \sum_{i=0}^{M-1} \log \left(1 - \frac{i}{N}\right)$$

Puisque $\log(1-x) = -x + x^2/2 + \dots$ nous avons

$$\log P = - \sum_{i=0}^{M-1} \frac{i}{N} + \sum_{i=0}^{M-1} \frac{i^2}{2N^2} + \dots$$

Calcul de P_{SAT} (3/4)

Si $N \gg M$, nous avons $M/N \ll 1$ et le terme principal dans cette somme sera

$$\log P = -\frac{M(M-1)}{2N} + O\left(\frac{M^3}{N^2}\right)$$

puisque $\sum_{i=0}^{M-1} i = \frac{M(M-1)}{2}$ d'après la formule de la somme arithmétique.

Par conséquent

$$P_{SAT} < P \approx \exp\left(-\frac{M(M-1)}{2N}\right) \approx \exp\left(-\frac{\alpha^2 N}{2}\right)$$

avec $\alpha = M/N$.

Calcul de P_{SAT} (4/4)

Nous remarquons que pour avoir une probabilité de satisfiabilité constante, il est nécessaire que M^2/N soit constant : c'est-à-dire que si nous multiplions le nombre de variables par 100, nous devons multiplier le nombre d'équations par 10 pour avoir la même difficulté de problème.

Problème 2-XORSAT

Les M équations formant le système peuvent être écrites comme

$$x_{i_1} + x_{i_2} = \nu$$

où $i_1, i_2 \in \{1, \dots, N\}$ et $i_1 \neq i_2$.

Une représentation par graphe (réseau) est possible pour ce système, avec les N variables x_i représentées comme des nœuds. Si deux nœuds apparaissent dans la même équation, il y a une arête les connectant, avec un poids binaire ν .

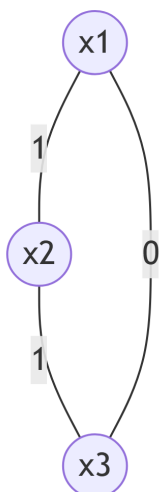
Le graphe associé au problème 2-XORSAT a $M = \alpha N$ arêtes, c'est-à-dire autant que d'équations.

Illustration

Le système

$$\begin{cases} x_1 + x_2 = 1 \\ x_2 + x_3 = 1 \\ x_1 + x_3 = 0 \end{cases}$$

admet la représentation graphique suivante :



Satisfiabilité et Cycles

Si le graphe obtenu est un arbre ou une forêt (une collection d'un ou plusieurs arbres disjoints), alors le système est satisfiable.

Il suffit de choisir arbitrairement la valeur de n'importe quelle feuille (c'est-à-dire la valeur correspondante x_i), et étape par étape d'affecter la valeur suivante en suivant l'arête.

- Si l'arête est 0, les deux nœuds prennent des valeurs identiques.
- Si l'arête est 1, ils prennent des valeurs opposées.

Si le système d'équations donne un graphe contenant des cycles, la satisfiabilité n'est pas garantie.

Exemple :

$$\begin{cases} x_1 + x_2 = 1 \\ x_2 + x_3 = 1 \\ x_3 + x_4 = 0 \\ x_4 + x_1 = 0 \end{cases}$$

En remontant les équations : $x_2 = \bar{x}_1$, $x_3 = \bar{x}_2 = x_1$, $x_4 = x_3 = x_1$, et finalement $x_1 = x_4 = x_1$ (cohérent). Il faut vérifier la parité des 1 sur le cycle.

Conditions pour la Satisfiabilité d'un Cycle

Il y a un nombre pair d'arêtes avec la valeur 1.

Comme le nombre de 1 sur un cycle est soit pair soit impair, il y a une probabilité $1/2$ que la satisfiabilité soit réalisée pour un cycle donné.

Conclusion :

La satisfiabilité d'un problème 2-XORSAT se réduit à estimer la probabilité qu'un graphe aléatoire ait des cycles avec un nombre impair de 1. Cela dépend clairement du nombre d'arêtes dans le graphe. Plus le nombre d'arêtes est élevé, plus le risque d'avoir un cycle brisant la condition de satisfiabilité est élevé.

Graphes Aléatoires : Erdős & Rényi

La probabilité de choisir une variable donnée comme point de départ d'une arête est de $1/N$.

La probabilité de choisir également cette même variable comme point d'arrivée d'une arête est aussi de $1/N$.

Ainsi, avec M arêtes tirées aléatoirement, le degré moyen d'un graphe aléatoire est $M(1/N + 1/N) = 2\alpha$.

- Si $2\alpha < 1$: le graphe pour le système 2-XORSAT comprend principalement des arbres indépendants. Ceci peut être compris par le fait que si $2\alpha < 1$, il y a en moyenne moins d'une arête par nœud, et donc une très faible probabilité de produire un cycle.
- Si $2\alpha > 1$: le graphe possède une composante géante qui contient $\mathcal{O}(N)$ nœuds. Il y a un grand nombre de cycles, et la probabilité qu'une solution existe diminue lorsque N augmente.

Ainsi, la satisfiabilité du problème varie avec le "paramètre de contrôle" α qui admet la valeur critique $\alpha_c = 1/2$.

Cas du Problème 3-XORSAT

Les problèmes 3-XORSAT correspondent à avoir M équations du type :

$$x_{i_1} + x_{i_2} + x_{i_3} = \nu$$

toutes contenant 3 variables booléennes parmi le total de N .

Si dans le système, certaines variables n'apparaissent qu'une seule fois dans une seule équation, alors le problème peut être simplifié.

Par exemple, si x_0 n'apparaît que dans l'équation

$$x_0 + x_i + x_j = \nu$$

alors nous pouvons déduire que

$$x_0 = \nu + x_i + x_j$$

ce qui nous permet d'éliminer x_0 du processus de résolution, ainsi que l'équation correspondante.

Élimination Itérative

Ce processus est itératif, dans le sens où après avoir éliminé une variable et l'équation correspondante, d'autres variables peuvent également n'apparaître qu'une seule fois dans une autre équation, qui peuvent à leur tour être éliminées.

Exemple :

$$\begin{cases} x_1 + x_2 + x_3 = \nu_1 \\ x_2 + x_4 + x_5 = \nu_2 \\ x_3 + x_4 + x_5 = \nu_3 \end{cases}$$

La variable x_1 peut être éliminée en premier en résolvant $x_1 = \nu_1 + x_2 + x_3$.

Il reste alors deux équations, à partir desquelles x_2 et x_3 peuvent être éliminées.

Finalement, il ne reste plus d'équation, et nous concluons que le problème est **satisfiable** (n'importe quelle valeur peut être choisie pour x_4 et x_5).

Réduction du Problème

- Par ce processus, nous pouvons simplifier le problème original S en un problème plus petit S' .
- À la limite des grands N , nous pouvons montrer avec une haute probabilité que S' est vide si

$$\alpha < \alpha_d = 0.8184$$

et par conséquent le problème est satisfiable.

- Pour $\alpha_d \leq \alpha < \alpha_c = 0.9179$: on peut montrer que S' n'est pas vide mais que certaines solutions existent.
- Pour $\alpha > \alpha_c$: il n'y a pas de solutions (toujours avec une haute probabilité et à la limite $N \rightarrow \infty$). Nous avons donc une transition de phase à $\alpha = \alpha_c = 0.9179$.

Structure de l'Espace des Solutions

Nous pouvons analyser comment les solutions peuplent l'hypercube de dimension N qui représente l'ensemble de toutes les valeurs possibles des x_i .

- Pour $\alpha < \alpha_d = 0.8184$: les solutions d'une instance du 3-XORSAT sont distribuées de manière homogène dans tout l'hypercube.
- Pour $\alpha_d \leq \alpha < \alpha_c = 0.9179$: les solutions sont regroupées en clusters sans aucune autre solution entre les clusters.

Comportement d'une Métaheuristique de Recherche

- Nous allons maintenant considérer le comportement d'une métaheuristique pour résoudre des instances du problème 3-XORSAT.
- **Note** : Les problèmes XORSAT peuvent être résolus en **temps polynomial** par élimination de Gauss et les métaheuristiques ne sont **PAS** un moyen efficace de trouver une solution !
- Cela dit :
 - Nous pouvons toujours analyser comment les métaheuristiques se comportent en faisant varier le niveau de difficulté du problème.

L'Algorithme Random Walk SAT (RWSAT)

L'algorithme Random Walk SAT (introduit par Papadimitriou en 1991) pour résoudre des instances du 3-XORSAT avec une métaheuristique basique.

Principe :

Choisir aléatoirement une affectation des N variables, puis lister toutes les équations qui ne sont pas satisfaites par ce choix initial.

Parmi ce sous-ensemble d'équations, nous en sélectionnons une au hasard et changeons la valeur d'une (et une seule) de ses 3 variables. Cela a pour conséquence de rendre cette équation satisfaite.

Évidemment, ce changement peut compromettre la validité d'autres équations qui étaient à l'origine satisfaites.

Nous répétons ce processus jusqu'à ce que toutes les équations soient satisfaites, ou jusqu'à ce que nous dépassions le temps de calcul maximum autorisé.

Pseudo-code de l'Algorithme RWSAT

Pour écrire un pseudo-code pour cet algorithme, nous introduisons la quantité E mesurant le nombre de contraintes non satisfaites. Le but est donc de **minimiser** E .

```
initialiser toutes les variables aléatoirement
calculer  $E$ 
 $t = 0$ 
tant que ( $E > 0$  et  $t < \max$ )
    choisir au hasard une équation non satisfaite
    choisir au hasard une de ses trois variables
    changer la valeur de cette variable en son complément
    calculer  $E$ 
     $t = t + 1$ 
fin tant que
afficher  $E$ ,  $t$ 
```

Comportement de la Métaheuristique : Régime des petits α

E diminue lentement et atteint la valeur 0.

L'algorithme RWSAT produit une solution.

Comportement de la Métaheuristique : Régime des grands α

E diminue mais $e = E/M$ fluctue autour d'une valeur de plateau strictement positive $e_{\text{plateau}} > 0$.

L'amplitude des fluctuations devient plus petite lorsque N grandit. Grâce à ces fluctuations, nous pouvons espérer atteindre $E = 0$.

Résumé du Comportement de la Métaheuristique

- Si $\alpha < \alpha_E = 0.33$: $E_p = 0$ et le temps de résolution croît linéairement avec la taille N du problème :

$$\langle T_{res} \rangle = N t_{res}$$

où t_{res} est un facteur qui croît avec α .

- Si $\alpha_E \leq \alpha < \alpha_c$: $E_p > 0$ et le temps de résolution croît exponentiellement avec N :

$$\langle T_{res} \rangle = \exp(N\tau_{res})$$

Il y a une transition de phase vers une complexité algorithmique exponentielle.

Complexité Algorithmique Linéaire et Exponentielle

Le graphique montre le temps de résolution moyen pour RWSAT. Pour mettre en évidence la tendance exponentielle, on trace $\log_{10} \langle T_{res} \rangle = f(N)$.

Relation entre $e_{plateau}$ et τ_{res} en fonction de α

Le graphique montre l'évolution de $e_{plateau}$ et τ_{res} en fonction de α .

Algorithme de Retour Arrière (Backtracking)

- Le défi avec l'algorithme RWSAT est la raréfaction des solutions dans l'espace de recherche.
- Ceci peut être compris en analysant le comportement de l'algorithme de retour arrière (backtracking) :
 - Les variables x_i sont progressivement affectées selon un arbre binaire qui est parcouru en profondeur.
 - Pour chaque nouvelle affectation, nous vérifions que la dernière affectation ne produit pas de conflit avec les équations pour lesquelles les variables sont déjà fixées.
 - En cas de contradiction, nous changeons la dernière valeur affectée (c'est-à-dire retour arrière).

Le temps de calcul est proportionnel au nombre de nœuds de l'arbre de recherche généré par l'algorithme de retour arrière avant de trouver une solution (affectation de tous les x_i).

Nous observons également une tendance avec un seuil (transition de phase) :

- Au-delà d'une valeur critique α_c (différente de celle de RWSAT), le retour arrière nécessite également un temps exponentiel pour résoudre le problème.
- Pour $\alpha < \alpha_c$, ce temps est linéaire.

$$\langle T_{res} \rangle = \begin{cases} Nt & \text{si } \alpha < \alpha_E \\ \exp(N\tau) & \text{si } \alpha > \alpha_E \end{cases}$$

Graphique de Performance

Performance de la recherche par retour arrière pour trouver une solution.

La valeur critique α_c dépend de la façon dont nous affectons, à chaque niveau de l'arbre, les variables x_i .

- Si cette affectation est effectuée aléatoirement (en parcourant aléatoirement les indices i), nous trouvons

$$\alpha_c = 2/3$$

- Si nous affectons d'abord les variables apparaissant dans les équations qui contiennent encore deux variables libres, nous trouvons

$$\alpha_c = 0.7507$$